SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO5: Analyze object-oriented software using quality assurance and usability testing techniques. (BL4,BL5)
Assessment Tool Weightage: 40%

QUESTIONS: 05 Total Marks:40 Annexure A

MOCK INTERVIEW

Code of Conduct:

I Sanjay J/192511331 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sanjay J

Annexure A

Mock Interview – Sample Questions (5 Questions)

Mock Interview Test Format for Object Oriented Analysis and Design (OOAD)

Section A Self Introduction & Fundamentals 3 Minutes 5 Section B Technical Questions (OOAD Concepts) 7 Minutes 10 Section C UML Modeling & Design 7 Minutes 10 Section D Case Study / Problem Solving 8 Minutes 10 Section E HR & Career-Oriented Questions 5 Minutes 5

SECTION A – SELF INTRODUCTION (5 Marks)

Evaluation Criteria

- Communication Skills
- Confidence
- Technical Awareness
- Academic Background

Sample Questions

1. Tell me about yourself.
2. Why did you choose Computer Science?
3. Explain your favorite subject in OOAD.
4. What software development tools have you used?
5. Describe a project where you applied object-oriented concepts.

SECTION B – TECHNICAL QUESTIONS (15 Marks)

Basic Level

Q1. What is Object-Oriented Analysis and Design?

Q2. Differentiate between Analysis and Design.

Q3. Explain Encapsulation with an example.
Q4. What is Inheritance? Mention its types.

Q5. Define Polymorphism and its benefits.

Q6. What is Abstraction?

Q7. Difference between Aggregation and Composition.

Q8. What is Coupling and Cohesion?

Q9. What is a Class and an Object?

Q10. Explain Object Responsibility.

SECTION C – UML MODELING (15 Marks)

Q11. What is UML?

Q12. When would you use a Use Case Diagram?

Q13. Draw and explain a Class Diagram for a Library Management

System. **Q14.**Differentiate Sequence Diagram and Activity Diagram.

Q15. Explain State Diagram with an example.

Q16. What is a Deployment Diagram?

Q17. How do Component Diagrams help software

architects? **Q18.**What is Multiplicity in UML?

SECTION D – CASE STUDY / PROBLEM SOLVING (10

Marks) Case Study 1

A hospital wants a system to manage:

- Patients
- Doctors
- Appointments
- Billing

Questions

1. Identify the classes.
2. Draw a simple class diagram.
3. Suggest relationships among classes.
4. Which UML diagrams would you use for analysis?
5. Explain how inheritance can be applied.

Case Study 2

An Online Shopping System supports:

- Customers
- Products
- Cart
- Orders
- Payments

Questions

1. Identify actors and use cases.
2. Design a use case diagram.
3. Suggest suitable design patterns.
4. Explain how polymorphism can be used in payment processing.

SECTION E – HR & PROFESSIONAL QUESTIONS (5

Marks) Q19. Why should we hire you?

Q20. What are your strengths and weaknesses?

Q21. How do you handle project deadlines?

Q22. Have you worked in a team project? Explain your

role. **Q23.**Where do you see yourself in five years?

RUBRICS FOR CALCULATION:

MOCK INTERVIEW

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively

Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

Q1. What is Object-Oriented Analysis and Design?

Object-Oriented Analysis and Design (OOAD) is a software engineering approach that models a system as a set of interacting objects.

• Object-Oriented Analysis (OOA): Focuses on understanding the system requirements and identifying domain objects ("what" the system does).

• Object-Oriented Design (OOD): Focuses on conceptualizing technical solutions and defining class structures, interfaces, and behaviors ("how" the system will be built). Q2. Differentiate between Analysis and Design.

Feature	Analysis	Design
Focus	Understanding the problem domain ("What").	Finding the technical solution ("How").
Orientation	User/Business perspective.	Technical/Implementation perspective.
Output	Use cases, requirement specs, domain models.	Class diagrams, design patterns, architecture specs.

Q3. Explain Encapsulation with an example.

Encapsulation is the practice of binding data (attributes) and methods that operate on that data into a single unit (a class) while restricting direct access to internal details. • Example: In a

BankAccount class, the balance variable is marked private. External modules cannot modify balance directly; they must call public methods like deposit() or withdraw(), ensuring validation rules are enforced.

Q4. What is Inheritance? Mention its types.

Inheritance is a mechanism where a child class derives attributes and methods from a parent class, enabling code reuse.

- Types:
    1. Single: One child inherits from one parent.
    2. Multiple: One child inherits from multiple parents.
    3. Multilevel: A class inherits from a derived class (chain).
    4. Hierarchical: Multiple children inherit from one parent.
    5. Hybrid: A combination of two or more inheritance types.

Q5. Define Polymorphism and its benefits.

Polymorphism ("many forms") allows objects of different classes to respond to the same method call in their own specific way.

- Benefits:
    - Increases code flexibility and extensibility.
    - Reduces code duplication.
    - Promotes loose coupling via interface-driven development.

Q6. What is Abstraction?

Abstraction is the concept of showing only the essential features of an object to the outside world while hiding the internal implementation details (e.g., using a car pedal without knowing engine mechanics).

Q7. Difference between Aggregation and Composition.

Feature	Aggregation	Composition
Association	Weak "HAS-A" relationship.	Strong "HAS-A" relationship.
Lifecycle	Child can exist independently of parent.	Child cannot exist without parent.
Example	Department and Teacher.	House and Room.

Q8. What is Coupling and Cohesion?

- Coupling: Measures the degree of interdependence between classes. Target: Low Coupling (fewer dependencies).
- Cohesion: Measures how focused and closely related the responsibilities within a single class are. Target: High Cohesion (one clear job).

Q9. What is a Class and an Object?

- Class: A blueprint or template defining attributes and methods.
- Object: An active instance of a class occupying memory space.

Q10. Explain Object Responsibility.

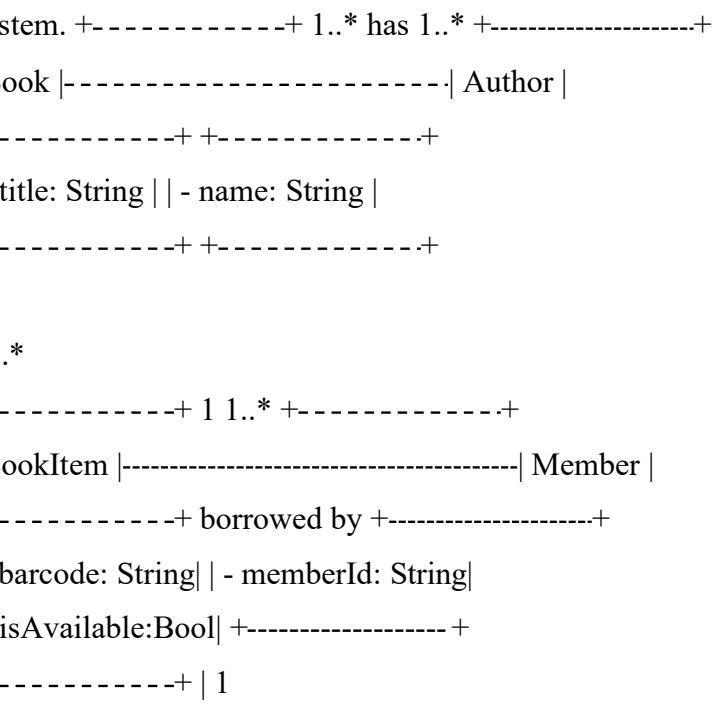
Object Responsibility defines the duties an object must perform. It falls into two categories: 1. Doing: Performing an action, creating objects, or managing calculations. 2. Knowing: Maintaining internal state data or knowing about related objects. SECTION C – UML MODELING

Q11. What is UML?

Unified Modeling Language (UML) is a standardized visual language used in software engineering to visualize, specify, construct, and document system architecture. Q12. When would you use a Use Case Diagram?

Use Case Diagrams are used during requirement gathering to identify external users (Actors) and the core functions (Use Cases) the system provides to them.

Q13. Draw and explain a Class Diagram for a Library Management



| 0..*

+- - - - - - - - - - - -+

| LoanRecord |

+- - - - - - - - - - - -+

| - issueDate: Date |

+- - - - - - - - - - - -+

• Explanation: Book holds general info and has multiple physical BookItem copies.

Member checks out BookItems, which generates LoanRecord entries.

Q14. Differentiate Sequence Diagram and Activity Diagram.

Feature	Sequence Diagram	Activity Diagram
Focus	Object interactions ordered in time.	Step-by-step workflow / business logic.

Feature	Sequence Diagram	Activity Diagram
Structure	Lifelines, activations, and messages.	Action nodes, decision diamonds, swimlanes.

Q15. Explain State Diagram with an example.

A State Diagram tracks the dynamic changes of a single object across its lifecycle in response to events.

• Example (ATM Session): Idle → *(Card Inserted)* → PIN Verification → *(Valid PIN)* →

Select Transaction → Dispensing Cash → Idle.

Q16. What is a Deployment Diagram?

A Deployment Diagram models the physical hardware nodes and execution environments (servers, containers, databases) where software components are deployed. Q17. How do Component Diagrams help software architects?

They allow architects to break down systems into modular software components, define required and provided interfaces, and evaluate system dependencies for build and deployment strategies.

Q18. What is Multiplicity in UML?

Multiplicity defines the valid range of instances allowed in a relationship between classes: • 1: Exactly one

• 0..1: Zero or one (optional)

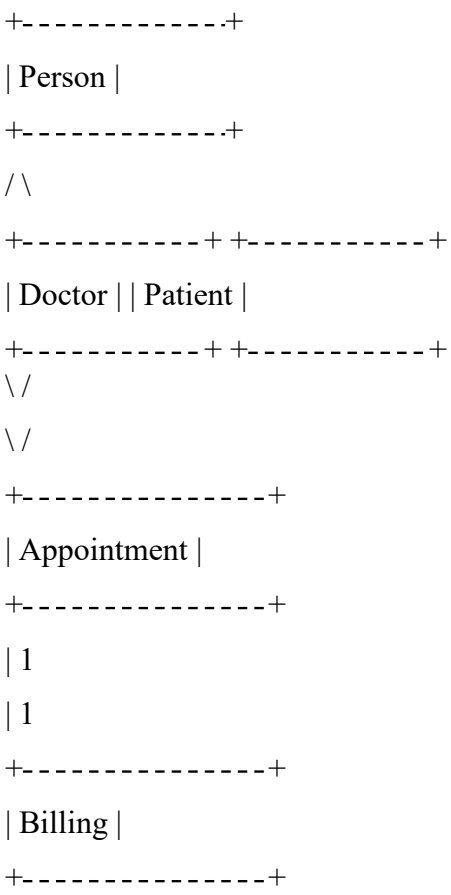
• * or 0..*: Zero or many

• 1..*: One or many

SECTION D – CASE STUDY / PROBLEM SOLVING

Case Study 1: Hospital Management System

1. Identified Classes: Person (Base), Doctor, Patient, Appointment, Billing.
2. Simple Class Diagram:

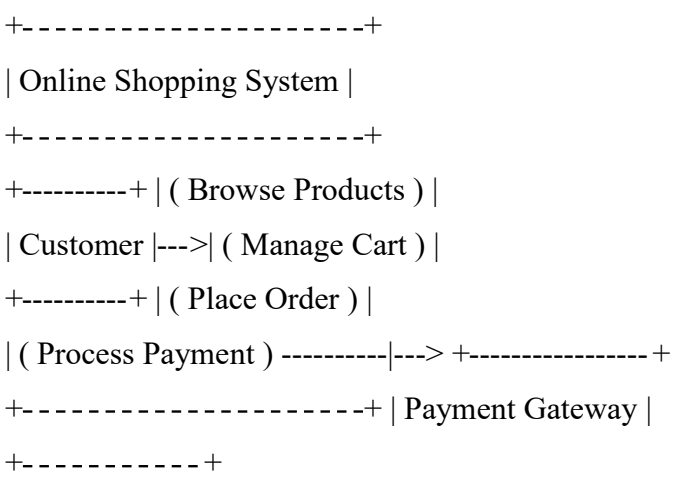


3. Relationships:
    - Inheritance: Doctor and Patient inherit from Person.
    - Association: Doctor and Patient interact through an Appointment.
    - Composition: Appointment owns a Billing record (strong link).
4. UML Diagrams for Analysis: Use Case Diagram (requirements analysis) and Sequence Diagram (appointment booking flow).
5. Applying Inheritance: Create an abstract base class Person (id, name, phone) and derive

Doctor (specialization, fee) and Patient (medicalHistory, insurance).

Case Study 2: Online Shopping System

1. Actors & Use Cases:
    - Actors: Customer, Admin, Payment Gateway.
    - Use Cases: *Browse Products*, *Manage Cart*, *Place Order*, *Process Payment*.
2. Use Case Diagram:



3. Design Patterns:
    - Strategy Pattern: For dynamic payment strategies (UPI, CreditCard, PayPal).
    - Observer Pattern: To send notifications when order status changes.
4. Polymorphism in Payment Processing: Create an interface PaymentStrategy with a pay(amount) method. Implementing classes (CreditCardPayment, UPIPayment) override pay() with custom implementations, allowing runtime resolution without changing order logic.

SECTION E – HR & PROFESSIONAL QUESTIONS

Q19. Why should we hire you?

I bring a solid foundation in software engineering, object-oriented design, and full-stack development, alongside hands-on experience building end-to-end applications and analyzing algorithms. I prioritize clean code, modular architecture, and rapid problem-solving, making me quick to add value to team goals.

Q20. What are your strengths and weaknesses?

• Strength: Strong analytical thinking and structured problem-solving skills, particularly in breaking complex system architectures into clean, maintainable modules. • Weakness: I sometimes spend extra time tweaking code structure or visual diagrams for perfection. To manage this, I set strict time-boxed goals to deliver on time. Q21. How do you handle project

deadlines?

I break down large requirements into prioritized tasks, estimate effort realistically, and focus on delivering core functional components first. If unexpected bottlenecks occur, I communicate early with stakeholders to align on trade-offs.

Q22. Have you worked in a team project? Explain your role.

Yes. In collaborative project environments, I have taken on roles managing technical architecture, designing database schemas, integrating APIs, and implementing backend logic. I regularly participate in code reviews and ensure clear system documentation. Q23. Where do you see yourself in five years?

In five years, I see myself as a Senior Software Engineer or Technical Architect, leading system design decisions, mentoring junior developers, and building scalable, high-performance.